



•OPERATING SYSTEM

•INSY 8223

Faculty of IT, GROUP D&F

Dr Eric NIZEYIMANA and Mr. Archimede IRAKOZE

eric.nizeyimana@auca.ac.rw | +250 783 308 544

Lesson 6. Memory Management

Learning Outcomes

At the end of this chapter, students should be able to:

- Demonstrate the understanding of memory management and its role in organizing and optimizing the use of computer memory for efficient execution of processes.
- Grasp the notion of process address space, exploring how programs utilize memory during program execution.
- Examine memory mapping techniques, gaining understanding of mechanisms used to map logical addresses to physical addresses.
- Understand the concept of swapping in memory management, recognizing how the operating system transfers portions of processes between main memory and secondary storage to optimize resource utilization.
- Explore different memory allocation techniques, including fragmentation, paging, and segmentation.
- Demonstrate the ability to choose appropriate methods based on system requirements to mitigate issues such as internal and external fragmentation.

What is memory management

Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution.

Memory management keeps track of each and every memory location, regardless of either it is allocated to some process or it is free.

It checks how much memory is to be allocated to processes. It decides which process will get memory at what time.

It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

Cont'

Memory management is a core function of the OS.

It ensures efficient allocation and deallocation of memory.

Prevents processes from interfering with each other.

Objectives of Memory Management

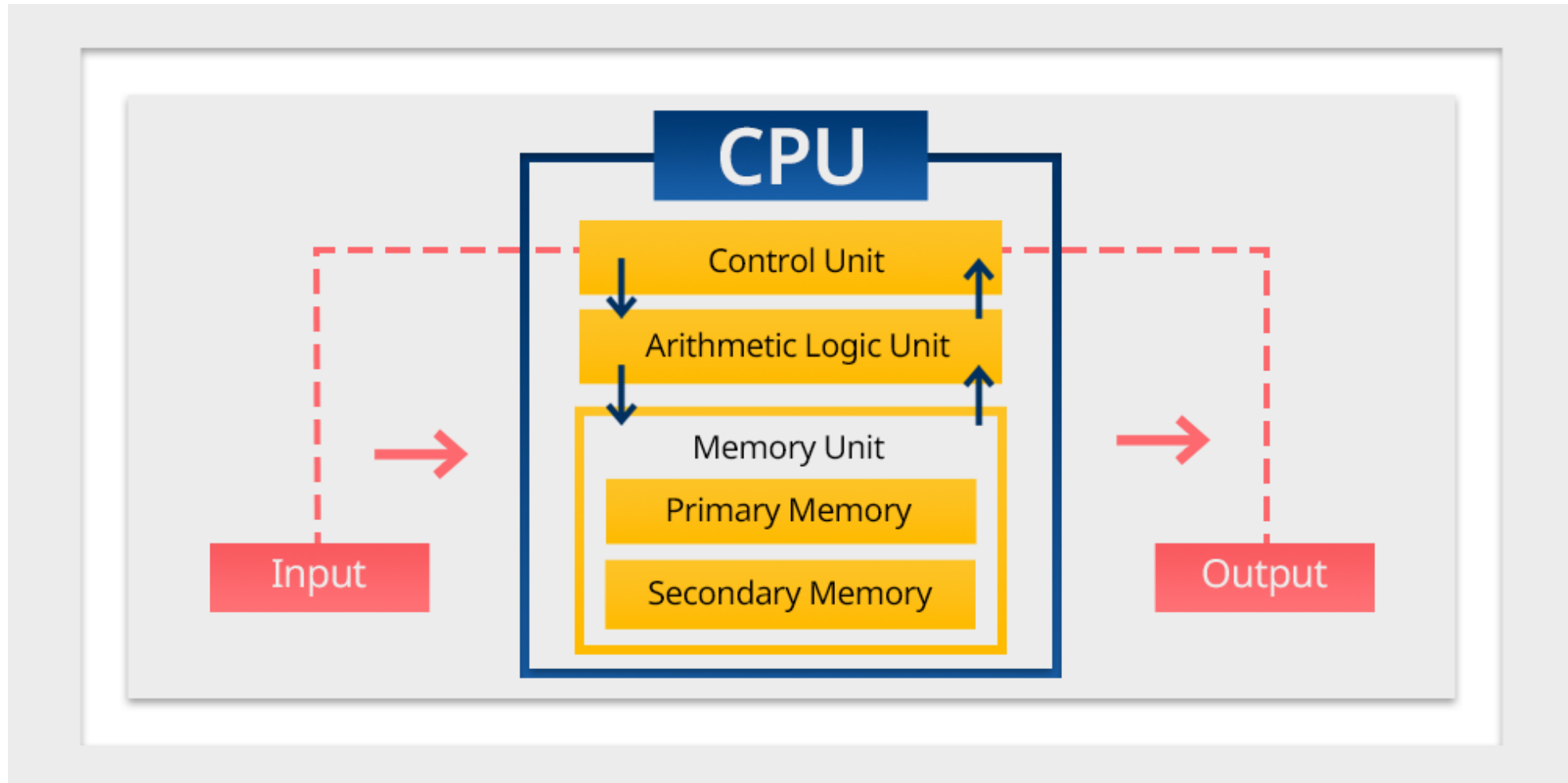
To keep track of memory usage.

To allocate memory to processes as needed.

To ensure system stability and multitasking.

To optimize CPU utilization.

Block diagram of CPU



Process Address Space

The process address space is the set of logical addresses that a process references in its code. For example, when 32-bit addressing is in use, addresses can range from 0 to 0x7fffffff; that is, 2^{31} possible numbers, for a total theoretical size of 2 gigabytes.

The operating system takes care of mapping the logical addresses to physical addresses at the time of memory allocation to the program. There are three types of addresses used in a program before and after memory is allocated .

Types of Addresses

S.N.	Memory Addresses & Description
1	Symbolic addresses The addresses used in a source code. The variable names, constants, and instruction labels are the basic elements of the symbolic address space.
2	Relative addresses At the time of compilation, a compiler converts symbolic addresses into relative addresses.
3	Physical addresses The loader generates these addresses at the time when a program is loaded into main memory.

Logical vs Physical address space

The set of all logical addresses generated by a program is referred to as a **logical address space**. The set of all physical addresses corresponding to these logical addresses is referred to as a **physical address space**.

The runtime mapping from virtual to physical address is done by the memory management unit (MMU) which is a hardware device.

Memory Management Responsibilities

Tracking memory – What part is free/occupied.

Process allocation – Assigning memory to programs.

Swapping – Moving processes in/out of memory.

Deallocation – Reclaiming memory after use.

Contiguous vs Non-Contiguous Allocation

Contiguous Allocation:

- Processes occupy a single continuous block.
- Simple but prone to **fragmentation**.

Non-Contiguous Allocation:

- Memory divided into chunks (paging/segmentation).
- More flexible and efficient.

Mapping from virtual to physical address

The value in the base register is added to every address generated by a user process, which is treated as offset at the time it is sent to memory. For example, if the base register value is 10000, then an attempt by the user to use address location 100 will be dynamically reallocated to location 10100.

The user program deals with virtual addresses; it never sees the real physical addresses.

Static vs Dynamic Loading

The choice between Static or Dynamic Loading is to be made at the time of computer program being developed. If you have to load your program statically, then at the time of compilation, the complete programs will be compiled and linked without leaving any external program or module dependency.

The linker combines the object program with other necessary object modules into an absolute program, which also includes logical addresses.

If you are writing a Dynamically loaded program, then your compiler will compile the program and for all the modules which you want to include dynamically, only references will be provided and rest of the work will be done at the time of execution.

Static vs Dynamic Loading (cont.)

At the time of loading, with **static loading**, the absolute program (and data) is loaded into memory in order for execution to start.

If you are using **dynamic loading**, dynamic routines of the library are stored on a disk in relocatable form and are loaded into memory only when they are needed by the program.

Static vs Dynamic Linking

As explained above, when static linking is used, the linker combines all other modules needed by a program into a single executable program to avoid any runtime dependency.

When dynamic linking is used, it is not required to link the actual module or library with the program, rather a reference to the dynamic module is provided at the time of compilation and linking. Dynamic Link Libraries (DLL) in Windows and Shared Objects in Unix are good examples of dynamic libraries.

Swapping

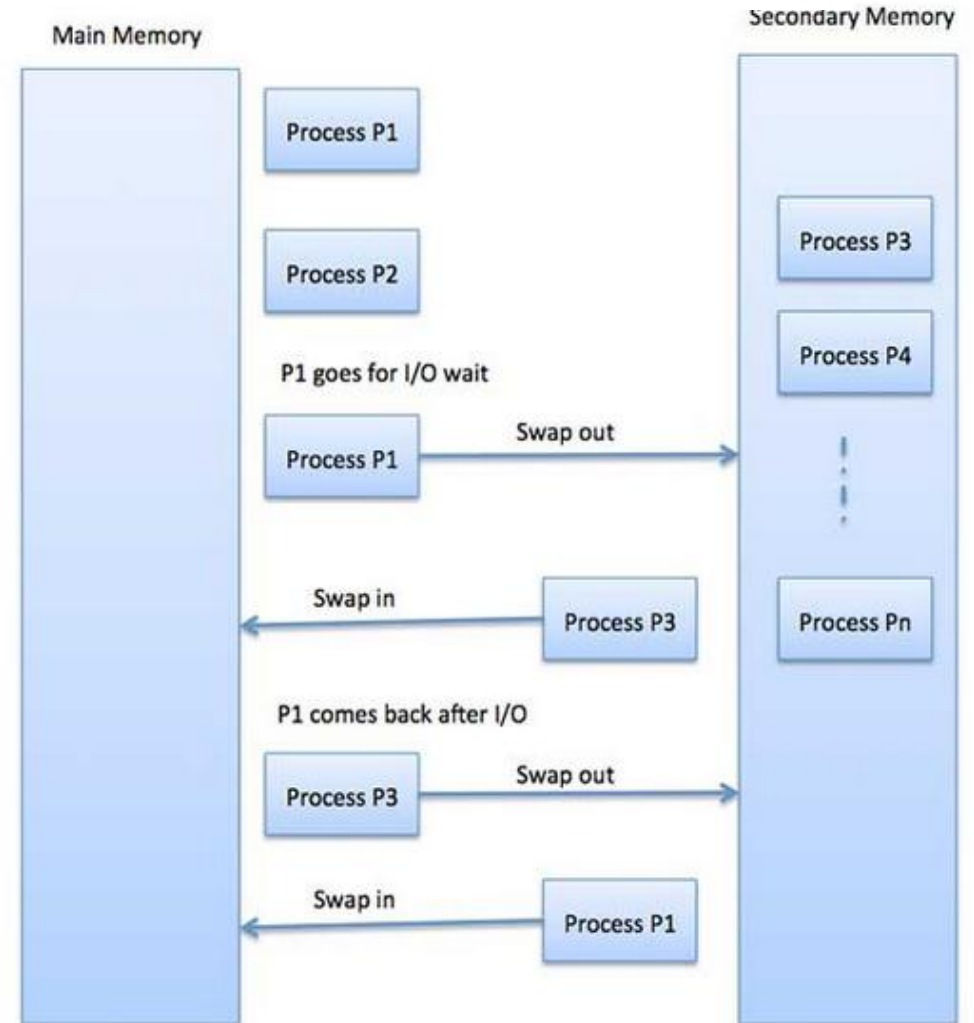
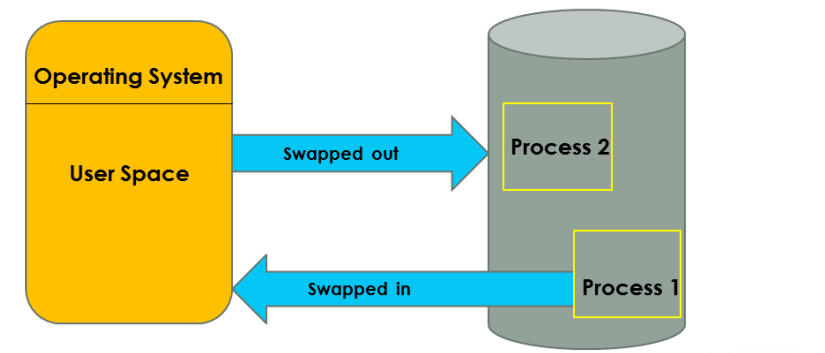
Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes. At some later time, the system swaps back the process from the secondary storage to main memory.

Though performance is usually affected by swapping process but it helps in running multiple and big processes in parallel and that's the reason **Swapping is also known as a technique for memory compaction.**

The total time taken by swapping process includes the time it takes to move the entire process to a secondary disk and then to copy the process back to memory, as well as the time the process takes to regain main memory.

Let us assume that the user process is of size 2048KB and on a standard hard disk where swapping will take place has a data transfer rate around 1 MB per second. The actual transfer of the 1000K process to or from memory will take

$2048\text{KB} / 1024\text{KB per second} = 2 \text{ seconds} = 2000 \text{ milliseconds}$



Disks

SSD: SSDs have much lower latency and faster read/write speeds because they use flash memory with no moving parts. Typical SATA SSDs can achieve read/write speeds of around 500–600 MB/s.

HDD: HDDs use spinning disks and mechanical read/write heads, leading to much higher latency and slower speeds, typically around 80–160 MB/s with a SATA connection.

Memory Allocation

Main memory usually has two partitions –

- **Low Memory** – Operating system resides in this memory.
- **High Memory** – User processes are held in high memory.

Memory allocation mechanism.

S.N.	Memory Allocation & Description
1	Single-partition allocation In this type of allocation, relocation-register scheme is used to protect user processes from each other, and from changing operating-system code and data. Relocation register contains value of smallest physical address whereas limit register contains range of logical addresses. Each logical address must be less than the limit register.
2	Multiple-partition allocation In this type of allocation, main memory is divided into a number of fixed-sized partitions where each partition should contain only one process. When a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.

Fragmentation

As processes are loaded and removed from memory, the free memory space is broken into little pieces. It happens after sometimes that processes cannot be allocated to memory blocks considering their small size and memory blocks remains unused. This problem is known as Fragmentation.

Types of Fragmentation

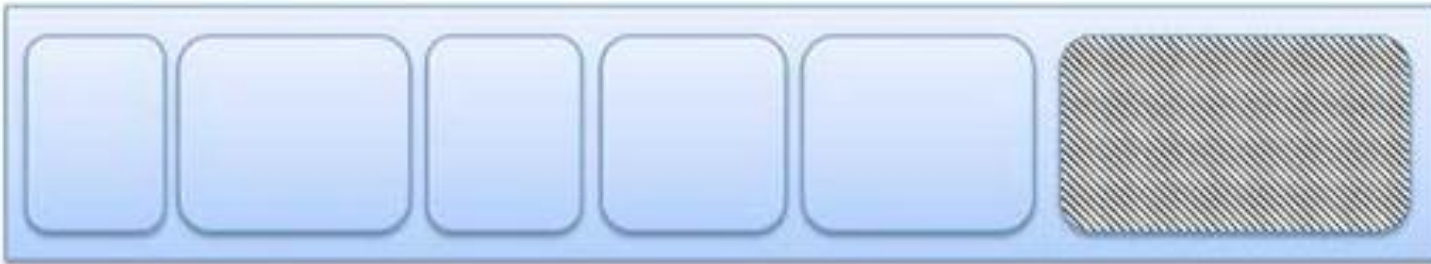
S.N.	Fragmentation & Description
1	External fragmentation Total memory space is enough to satisfy a request or to reside a process in it, but it is not contiguous, so it cannot be used.
2	Internal fragmentation Memory block assigned to process is bigger. Some portion of memory is left unused, as it cannot be used by another process.

The following diagram shows how fragmentation can cause waste of memory and a compaction technique can be used to create more free memory out of fragmented memory –

Fragmented memory before compaction



Memory after compaction



External fragmentation can be reduced by compaction or shuffle memory contents to place all free memory together in one large block. To make compaction feasible, relocation should be dynamic.

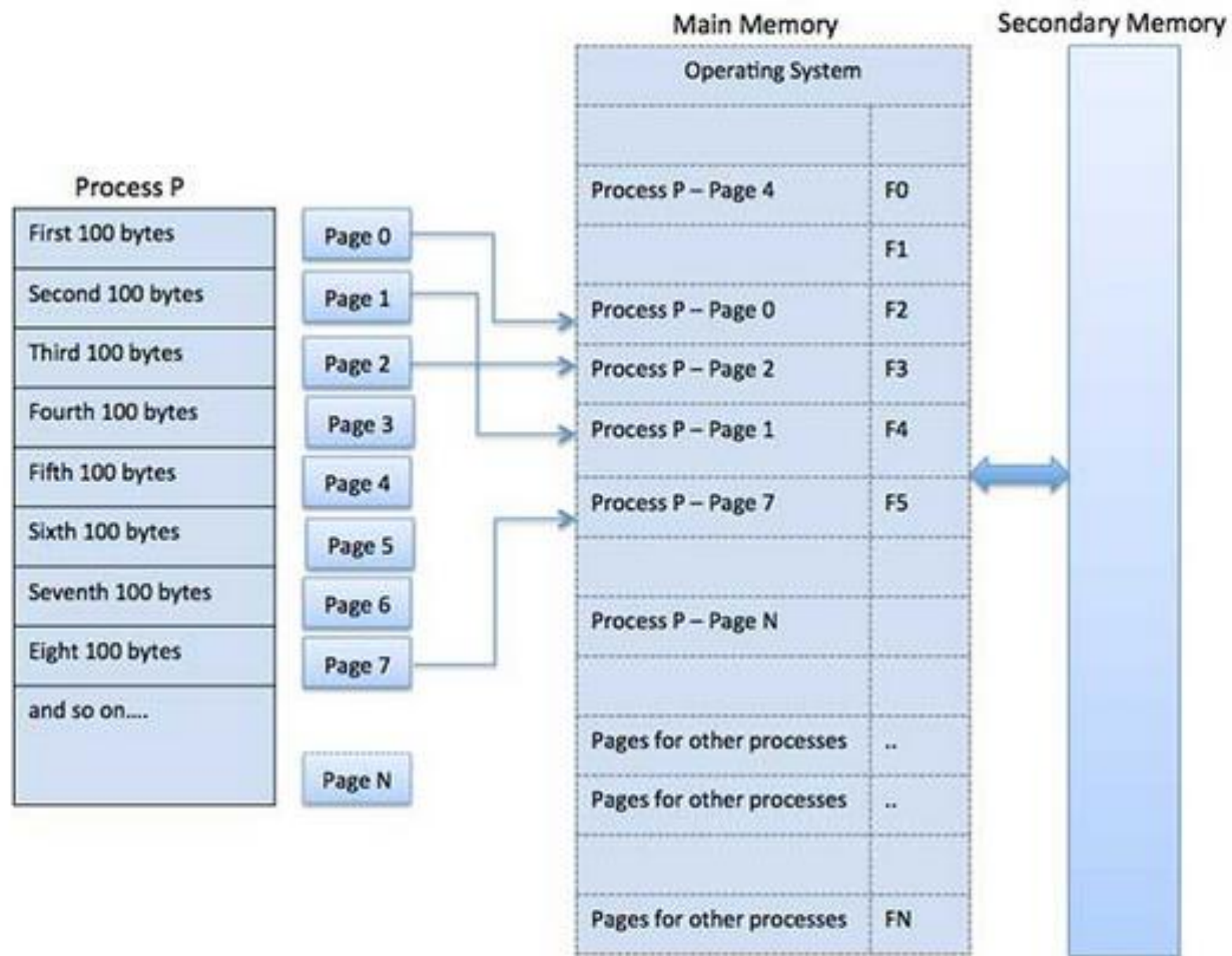
The internal fragmentation can be reduced by effectively assigning the smallest partition but large enough for the process.

Paging

A computer can address more memory than the amount physically installed on the system. This extra memory is actually called virtual memory and it is a section of a hard disk that's set up to emulate the computer's RAM. Paging technique plays an important role in implementing virtual memory.

Paging is a memory management technique in which process address space is broken into blocks of the same size called pages (size is power of 2, between 512 bytes and 8192 bytes). The size of the process is measured in the number of pages.

Similarly, main memory is divided into small fixed-sized blocks of (physical) memory called frames and the size of a frame is kept the same as that of a page to have optimum utilization of the main memory and to avoid external fragmentation.



Address Translation

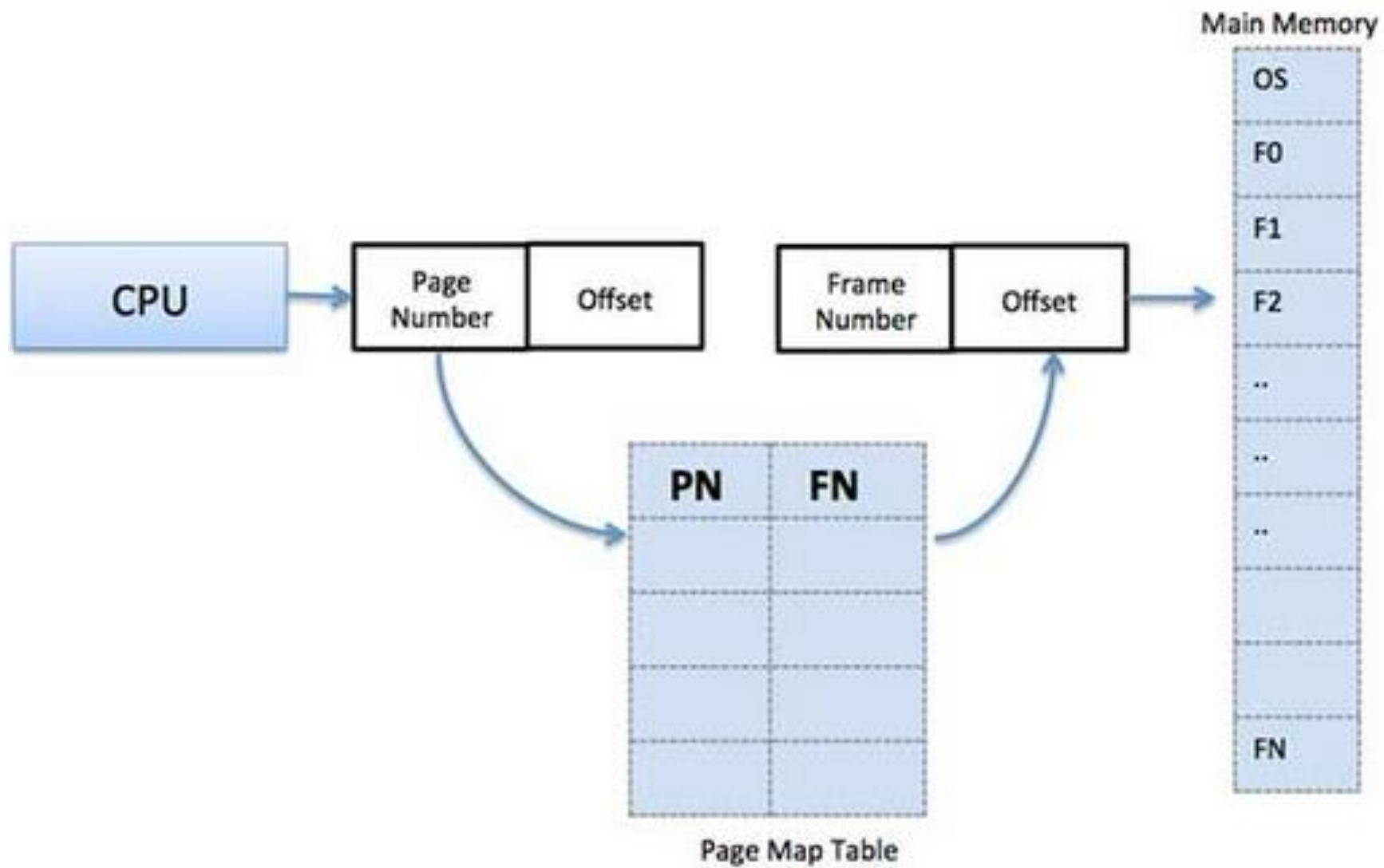
Page address is called **logical address** and represented by page number and the offset.

- Logical Address = Page number + page offset

Frame address is called **physical address** and represented by a frame number and the offset.

- Physical Address = Frame number + page offset

A data structure called **page map table** is used to keep track of the relation between a page of a process to a frame in physical memory.



Address Translation (cont.)

When the system allocates a frame to any page, it translates this logical address into a physical address and create entry into the page table to be used throughout execution of the program.

When a process is to be executed, its corresponding pages are loaded into any available memory frames. Suppose you have a program of 8Kb but your memory can accommodate only 5Kb at a given point in time, then the paging concept will come into picture. When a computer runs out of RAM, the operating system (OS) will move idle or unwanted pages of memory to secondary memory to free up RAM for other processes and brings them back when needed by the program.

This process continues during the whole execution of the program where the OS keeps removing idle pages from the main memory and write them onto the secondary memory and bring them back when required by the program.

Advantages and Disadvantages of Paging

Paging reduces external fragmentation, but still suffer from internal fragmentation.

Paging is simple to implement and assumed as an efficient memory management technique.

Due to equal size of the pages and frames, swapping becomes very easy.

Page table requires extra memory space, so may not be good for a system having small RAM.

Segmentation

Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions. Each segment is actually a different logical address space of the program.

When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory.

Segmentation memory management works very similar to paging but here segments are of variable-length whereas in paging pages are of fixed size.

A program segment contains the program's main function, utility functions, data structures, and so on. The operating system maintains a **segment map table** for every process and a list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory. For each segment, the table stores the starting address of the segment and the length of the segment. A reference to a memory location includes a value that identifies a segment and an offset.

Process P



Segment Map table

SN	Size	Memory Address
1	400	100
2	200	500
3	100	800
N	x	NM

Main Memory

Operating System
100
200
300
400
500
600
700
800
NM

While segmentation splits the program into segments of varying sizes, paging divides memory into fixed-size blocks known as pages. Modern operating systems commonly mix paging and segmentation to efficiently manage memory, as both techniques have advantages and disadvantages.

S. NO	Paging	Segmentation
1.	In paging, the program is divided into fixed or mounted size pages.	In segmentation, the program is divided into variable size sections.
2.	For the paging operating system is accountable.	For segmentation compiler is accountable.
3.	Page size is determined by hardware.	Here, the section size is given by the user.
4.	It is faster in comparison to segmentation.	Segmentation is slow.
5.	Paging could result in internal fragmentation.	Segmentation could result in external fragmentation.
6.	In paging, the logical address is split into a page number and page offset.	Here, the logical address is split into section number and section offset.
7.	Paging comprises a page table that encloses the base address of every page.	While segmentation also comprises the segment table which encloses segment number and segment offset.

8.	The page table is employed to keep up the page data.	Section Table maintains the section data.
9.	In paging, the operating system must maintain a free frame list.	In segmentation, the operating system maintains a list of holes in the main memory.
10.	Paging is invisible to the user.	Segmentation is visible to the user.
11.	In paging, the processor needs the page number, and offset to calculate the absolute address.	In segmentation, the processor uses segment number, and offset to calculate the full address.
12.	It is hard to allow sharing of procedures between processes.	Facilitates sharing of procedures between the processes.
13	In paging, a programmer cannot efficiently handle data structure.	It can efficiently handle data structures.
14.	This protection is hard to apply.	Easy to apply for protection in segmentation.
15.	The size of the page needs always be equal to the size of frames.	There is no constraint on the size of segments.
16.	A page is referred to as a physical unit of information.	A segment is referred to as a logical unit of information.
17.	Paging results in a less efficient system.	Segmentation results in a more efficient system.

Exercises in class

- **Make a group of two students and then do following exercises:**

1. Draw a memory layout showing how a system allocates memory to OS, processes, stack, heap, and free space.

Explain how your drawing ensures the system operates at optimal performance.

- **Requirements:**

- Label OS/Kernel area
- Label process P1 and P2 memory regions
- Include stack and heap
- Show fragmentation (if any)
- Explain how the layout promotes optimal performance (e.g., less fragmentation, efficient allocation, contiguous free space)

Lesson 7. File Systems and Disk Management

Thank you



For I know the thoughts that I think toward you, saith the LORD, thoughts of peace, and not of evil, to give you an expected end. Jeremy 29:11